claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a387a47e1f5337b7d.jsonl`
- SHA-256: `3cc7baef45fcd50b0697c267bbd1294229a3a3196f18ef5ead52bcc6a18e5bbf`
- Source modified: `2026-07-06T19:19:49+00:00`
- Imported at: `2026-07-08T16:00:11+00:00`
- project: `wf_e1b943dd-dac`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T19:19:01.870Z)

```
Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR
(branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F
design doc for a
four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content
safety / Tier 4
deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up
issues F1-F7.
No code changes. Read the ADR:
  git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-
pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design
contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the
codebase and the
other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C
C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem,
Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate
should_ingest_source,
delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler:
watched gate -> dedup
-> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track
C (delivery_attempts)
just merged.

Ground rules:
- READ actual files, cite ADR section / file:line. You MAY run
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does
FilterConfig
  ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text
populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies,
not merge-blockers.
  Severity: high (a claim that is false, or a design gap that would break an F-phase / cause
data loss), medium
  (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example,
minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for
implementation.
```

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if
unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (cross_adr): Tier 3 "Duplicate suppression" stage overlaps Track B/E dedup that already runs before the pipeline, and its "cross-policy" claim has no supporting schema
severity low | section Security stages ? Tier 3, item 3 ("Duplicate suppression")
summary: Tier 3 adds a "duplicate suppression" stage that "adds cross-policy duplicate detection within an account's delivery streams," while admitting content dedup is "already handled by content_items.dedup_key." The handler already does three-tier dedup before the pipeline, and the ADR 0011 UNIQUE(account_id, dedup_key) index enforces per-account dedup at the DB level. The stage's only novel claim (cross-policy/per-destination duplicate suppression) has no schema to support it in this ADR.
why_it_matters: As written the stage is mostly redundant with pre-pipeline dedup; the one non-redundant behavior (suppress same content across multiple destinations/policies) would need delivery_attempts-based per-destination checks that aren't specified. Minor because it's clearly a stub, but the redundancy should be acknowledged so F-phase authors don't reimplement existing dedup.
evidence: telethon_client.py:192-243 performs three-tier dedup (document_id, fingerprint, message instance) BEFORE evaluate/insert/deliver. ADR 0011 content_items has UNIQUE(account_id, dedup_key). ADR 0012 text: "already handled by content_items.dedup_key; this stage adds cross-policy duplicate detection" ? but no table/index/query is defined for the cross-policy case.
rec: Either drop the stage (dedup is a pre-pipeline concern owned by Track B) or specify the cross-destination mechanism concretely (e.g. "suppress if a delivered delivery_attempts row already exists for this content_item across the account's destinations"), and clarify it is delivery-time de-duplication, not content dedup.

2. assistant (2026-07-06T19:19:04.425Z)

I'll verify this finding by reading the ADR section and the referenced code/ADRs.

3. user (2026-07-06T19:19:06.099Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig`
(per account, via `policies.config_json`). The filter is **flat**: all rules
run unconditionally, the output is a boolean (`matched`), and there is no
way to express topic preferences, security policy, or why a specific video
passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that
run before delivery ? so the system can express richer decisions, explain
them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
`FilterConfig` / `Policy`	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.

`FilterEngine`	Evaluates all rules; returns `FilterResult(passed, reasons)`.
`policies` table	One row per account (is_default=1). `config_json` is the `FilterConfig` blob.
Security / topic / deep-inspect	Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan.

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
????????????????????????????????????????????????????????????????
?                    Policy Pipeline                           ?
?                                                              ?
?  Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?     duration, resolution, size, codec, mime, GIF/round       ?
?     ? extends today's FilterEngine                          ?
?                                                               ?
?  Tier 2 ? Topic preferences  (opt-in, metadata-only)         ?
?     source allowlist, keyword match, channel topic tags      ?
?     ? user-expressed interests filter the stream             ?
?                                                              ?
?  Tier 3 ? Content safety  (opt-in, metadata + light I/O)     ?
?     URL/domain reputation, file metadata sanity,             ?
?     caption text scanning                                    ?
?     ? cheap checks; no download                              ?
?                                                              ?
?  Tier 4 ? Deep inspection  (opt-in, heavy I/O)               ?
?     ffprobe, perceptual hash, downloaded content scan        ?
?     ? gated behind explicit user opt-in + budget             ?
????????????????????????????????????????????????????????????????
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem        # from ADR 0011
    source: Source                   # the source it came from
    policy: Policy                   # the owning policy config

@dataclass
class StageResult:
    passed: bool
    reason: str                      # human-readable explanation
    stage_name: str                  # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict)  # for explainability UI
```

```
class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any"  // "any" = match any include topic; "all" = match all
  }
}
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider_source_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

Policy evaluation flow (post-dedup, pre-delivery)

```python
async def evaluate_pipeline(
```

```
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results  # short-circuit
    return True, results
```

The caller (telethon_client or future Bot API handler) records the
results via `delivery_attempts.last_error` (for failed items) or a new
`policy_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

No schema migration is required for this design. The `policies` table
already holds a `config_json` JSON blob. The pipeline is a code concept:

- `policies.config_json` gains optional `topics`, `allowlist`,
  `security_checks`, `deep_inspect` keys ? all backwards-compatible with
  the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy_decisions` table (deferred) would store the explanation
  trace for debugging: `(content_item_id, policy_id, stage_name, passed,
  reason, metadata_json, evaluated_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
F1 ? Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler	None
F2 ? Topic preferences	`topics` config parsing, source title/caption keyword matching	F1
F3 ? Source allowlist	`allowlist` config, skip non-allowlisted sources	F1
F4 ? URL/domain reputation	Domain blocklist, caption link extraction	F1
F5 ? Deep inspection	ffprobe/perceptual hash, download budget enforcement	F1, ADR 0003 waiver
F6 ? Policy decisions table	Audit trail storage, explainability UI	F1
F7 ? Explainability endpoint	`/why <content_item_id>` command showing the explanation trace	F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR
only defines the **design contract** the phases code against.

Consequences

Positive
- Unifies topic, security, and quality gates into a single explainable
  framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for
  items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without
  additional DB lookups at decision time.
- Backwards-compatible: `policies.config_json` already exists; adding
  optional keys requires no migration.
- Each implementation phase is independently shippable behind feature
  flags.

Negative
- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same

```
  interface, which may be awkward for deeply different checks
  (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict
  catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not
  semantic. A future ML stage would replace it, but it's good enough for
  a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a
  budget to avoid runaway costs.
```

Alternatives considered

```
- **Do nothing; keep flat FilterConfig.** Rejected ? the umbrella exists
  to dissolve the conflation of quality, topic, and security into a
  single boolean. A pipeline is the natural generalization.
- **Inline each stage in the handler** (no protocol). Rejected ? makes
  it impossible to test stages independently, reorder them, or add new
  ones without touching the hot path.
- **Use a rules engine / DSL.** Rejected ? over-engineered for <100
  users. A Python list of callables is simpler, testable, and
  introspectable.
- **Bundle all phases into a single PR.** Rejected ? scope discipline.
  Each phase is independently shippable; this ADR defines the shared
  interface they code against.
```

Open questions for the decision owner

```
1. **`StageResult.metadata` shape** ? dict or typed per-stage? Recommend:
   dict for Phase 1 (flexible), typed in Phase 2 (once the shape
   stabilizes).
2. **Policy decisions table** ? store every decision, or only failures?
   Recommend: every decision (explainability), with a TTL-based cleanup
   for old traces.
3. **Topic matching** ? substring only, or regex/glob? Recommend:
   substring (case-insensitive) for v1; natural-language matching in a
   follow-up.
4. **Deep inspection budget** ? per-item byte limit or per-account
   bandwidth cap? Recommend: per-item limit (10 MB default) with a
   per-account monthly cap enforced at the app level.
```

References

```
- ADR 0003 ? metadata-first probing (the no-download principle that Tier
  4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept
  generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this
  design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.
```

4. assistant (2026-07-06T19:19:09.936Z)

```
Now let me read the referenced code and ADR 0011 to verify the dedup claims.
```

5. user (2026-07-06T19:19:11.568Z)

```
        channel_id = _peer_id(message.peer_id)
        message_id = message.id
        document_id = _extract_document_id(message)

        # --- watched gate (ADR 0009, Track E) -------------------------------
        # Opt-in: a source is ingested only when watched=True. An absent
        # sources row means the channel has never been seen ? treat it as
```

```
# unwatched and lazily create a watched=0 row for /channels
# discoverability (ADR 0009 lines 37-44, 130-131).
if not await db.should_ingest_source(conn, user.id, channel_id):
    return  # skip unwatched / unknown source

# --- three-tier dedup (cheapest first) ---

# Tier 0: same (user, channel, message) instance ? already processed.
existing = await db.get_video(conn, user.id, channel_id, message_id)
if existing is not None:
    return

# Tier 1: same Telegram document.id ? forwarded/cross-posted file.
if document_id is not None:
    existing = await db.get_video_by_document_id(conn, user.id, document_id)
    if existing is not None:
        log.info(
            "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
            "original_channel=%s original_msg=%s",
            user.id,
            document_id,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
        return

# Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
if (
    meta.size_bytes is not None
    and meta.duration_s is not None
    and meta.height is not None
    and meta.width is not None
):
    existing = await db.get_video_by_content_fingerprint(
        conn,
        user.id,
        meta.size_bytes,
        meta.duration_s,
        meta.height,
        meta.width,
    )
    if existing is not None:
        log.info(
            "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
            "channel=%s msg=%s original_channel=%s original_msg=%s",
            user.id,
            meta.size_bytes,
            meta.duration_s,
            meta.height,
            meta.width,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
        return

# Lazily record the channel (best-effort; failures are non-fatal).
try:
    await db.upsert_channel(
        conn,
        Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
```

6. user (2026-07-06T19:19:12.815Z)

```
29:The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
32:- Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
43:### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR 0006?
45:The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
46:concept). A provider-neutral `ContentItem` needs a dedup story that works
52:`dedup_key` column on `content_items`:
55:content_items.dedup_key TEXT NOT NULL
58:The *value* of `dedup_key` is computed by an explicit decision tree at
64:  1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
65:  2. Else if **all four** fingerprint fields (`size_bytes`, `duration_s`,
66:     `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
67:  3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
69:     `message_id` is only unique within a channel, not across sources).
70:- **Future WhatsApp:** `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
71:  ids are globally unique per account) or `"wa:sha256:{blob_hash}"` if WA
76:`compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
81:A **partial unique index** on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
82:replaces the current Tier-1 index and enforces per-account dedup at the DB
90:- Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
96:- The migration must compute `dedup_key` for every existing `videos` row
98:- The `dedup_key` derivation logic lives in code (a pure function per
104:`dedup_key = sha256(downloaded_content)`. Cryptographically strong,
114:`dedup_key = "tg:doc:{document_id}"` always; no metadata fingerprint
115:fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new
119:**Recommendation: 1A.** It preserves the dedup quality of ADR 0006, generalizes cleanly, and
honors the no-download constraint.
141:    is_default   INTEGER NOT NULL DEFAULT 0,        -- one per account (UNIQUE partial idx)
144:    UNIQUE (account_id, name)
205:    UNIQUE (account_id, provider, provider_source_id)
237:    dedup_key         TEXT NOT NULL,         -- ADR 0011 Question 1 (Option 1A)
240:    -- Tier-0 equivalent: provider_message_id is only unique within a source
243:    UNIQUE (account_id, source_id, provider_message_id),
244:    UNIQUE (account_id, dedup_key)                        -- Tier-1/2 equivalent (Option 1A)
274:    UNIQUE (account_id, provider, target)
292:    UNIQUE (account_id, source_id, destination_id)
298:Per #28's idempotency requirement (unique index on
313:    UNIQUE (content_item_id, destination_id)         -- ADR #28 idempotency
331:3. **Backfill `content_items`** from `videos`, computing `dedup_key` via the
335:     `dedup_key = 'tg:doc:' || document_id`.
337:     `dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
width`.
339:     `dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id`,
344:   each branch (including all-fingerprint-fields-null) and assert no two
345:   rows collapse to the same `dedup_key`.
365:- `dedup_key` (Option 1A) generalizes ADR 0006 without losing any dedup
375:- The dedup-key derivation logic is in code, not the schema ? needs its
376:  own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
393:  views don't give us the `provider` column or the `dedup_key` index, and
398:1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
414:- ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
```

7. assistant (2026-07-06T19:19:15.224Z)

Now let me check the delivery_attempts table schema (idempotency) and the delivery_attempts unique index that would be relevant to per-destination dedup.

8. user (2026-07-06T19:19:17.568Z)

`delivery_attempts` (new ? Track C will populate)

Per #28's idempotency requirement (unique index on `(content_item_id, destination_id)`).

```
delivery_attempts (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    content_item_id INTEGER NOT NULL REFERENCES content_items(id),
    destination_id  INTEGER NOT NULL REFERENCES destinations(id),
    route_id        INTEGER REFERENCES routing_rules(id),
    status          TEXT NOT NULL DEFAULT 'pending',  --
'pending'|'delivered'|'failed'|'retrying'
    attempt_count   INTEGER NOT NULL DEFAULT 0,
    last_error      TEXT,
    created_at      TEXT NOT NULL,
    delivered_at    TEXT,
    next_retry_at   TEXT,
    UNIQUE (content_item_id, destination_id)         -- ADR #28 idempotency
);
```

Migration strategy (integer-preserving, phased)

This track lands **`sources`, `content_items`, and `policies`** only.
`destinations`, `routing_rules`, `delivery_attempts` are defined in the

9. assistant (2026-07-06T19:19:35.885Z)

I now have enough to assess this finding. Let me evaluate.

The finding claims:
1. Tier 3 "Duplicate suppression" overlaps pre-pipeline dedup ? **TRUE**:
telethon_client.py:192-243 does three-tier dedup before evaluate/insert/deliver, and ADR 0011
has `UNIQUE(account_id, dedup_key)`.
2. The stage's only novel claim (cross-policy/per-destination) has no supporting schema in this
ADR ? **This is where I need to scrutinize.**

Let me re-read the exact ADR 0012 text: *"Duplicate suppression: enhanced dedup that compares by
`content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage
adds cross-policy duplicate detection within an account's delivery streams)."*

Key observations for adversarial refutation:

- The ADR **explicitly acknowledges** the redundancy: "already handled by
`content_items.dedup_key`". So the finding's rec ("the redundancy should be acknowledged") is
partly already satisfied ? the ADR does acknowledge it. This weakens the finding.
- The novel behavior is "cross-policy duplicate detection within an account's delivery streams."
The finding says there's no schema to support it. Is that accurate? ADR 0011's
`delivery_attempts` has `UNIQUE(content_item_id, destination_id)` and status='delivered'. That
does provide a substrate: you could query delivered rows across destinations for a
content_item. But ADR 0012 does not reference `delivery_attempts` for this stage nor specify the
mechanism. So "no schema/query is defined *for the cross-policy case* in this ADR" is literally
accurate ? the ADR names no table/index/query for it.

However ? this is a Tier 3 stage whose implementation is explicitly deferred ("implementation
deferred to separate issues"). ADR 0012 has no F-phase for "duplicate suppression" at all. Let
me verify: F1-F7 phases ? F3 allowlist, F4 URL reputation, F5 deep inspect. There is **no
F-phase for duplicate suppression**. So the stage is listed in Tier 3 prose but never phased.
That actually strengthens the "it's a stub" characterization but also means no F-phase would
break.

Let me finalize the severity assessment. This is a draft design doc. The finding is: a
redundant/underspecified stage. It's not a false claim (the ADR's claim that dedup is "already
handled by dedup_key" is true), not data loss. The one imprecision: it claims to add cross-
policy detection without specifying mechanism. That's a genuine minor gap/vagueness. The ADR
does self-acknowledge the redundancy, which the finding's rec asks for ? partial refutation of
the rec.

Verdict: CONFIRMED but genuinely low. The gap (unspecified cross-policy mechanism, no phase, redundancy with pre-pipeline dedup) is real. It's not refuted because the ADR names no mechanism/schema/query/phase for the novel behavior. Severity stays low ? it's a clearly-labeled stub in a design doc, no F-phase depends on it, no false claim, no data loss.

10. user (2026-07-06T19:19:49.323Z)

Structured output provided successfully